Lecture 03 - Part I

Virtualization

From Popek–Goldberg to modern VMM implementations

High Performance and Cloud Computing 2025/2026
Applied Data Science & Artificial Intelligence - HPC and Cloud
University of Trieste

Learning Objectives: Part I

By the end of Part I, the student will be able to…

1. Define precisely what a virtual machine and a VMM are.
2. Enumerate the three VMM requirements (equivalence, safety, performance) and explain their role.
3. Identify the minimal hardware prerequisites for running a protected OS.
4. Explain the direct-execution / trap-and-emulate model and the deprivileging of the guest OS.
5. Classify instructions as privileged, sensitive (control- or behavior-), or regular.
6. State the Popek–Goldberg theorem and apply it to diagnose a real ISA.
7. Explain binary translation, paravirtualization, and HW-assisted virtualization as three responses to Popek–Goldberg.

Outline Part I

A single narrative arc: from motivation to the three responses

1. **Motivation and history** *~5 min*
2. **Simplified hardware model** *~10 min*
3. **Virtual machines and the three requirements** *~8 min*
4. **Direct execution and trap-and-emulate** *~10 min*
5. **Does trap-and-emulate always work?** *~8 min*
6. **Two concrete examples: HLT and popf** *~10 min*
7. **The Popek–Goldberg theorem** *~5 min*
8. **Three historical answers: BT / Paravirt / HW-assisted** *~12–15 min*

1 Motivation and history

Why do we care? A six-decade story.

Why virtualization? The problem before

Server sprawl and the utilization crisis

The problem (1990s–early 2000s)

- One application per physical server
- Average CPU utilization: 10–15%
- Each new workload → buy new hardware
- "Works on my machine" dependency hell
- Power, cooling, floor space exploding

What virtualization promises

- Multiple workloads on one physical host
- 60–80% sustained utilization
- Workload isolation within the same host
- Faster provisioning (minutes, not months)
- **The precondition for cloud computing**

Key observation: *cloud computing revived mainframe-era time-sharing on commodity x86. Virtualization is the enabling technology.*

A brief history of virtualization

Six decades in six milestones

1967	1998	2003	2006	2008	2017
IBM CP-40	**VMware**	**Xen**	**Intel VT-x**	**KVM merged in Linux**	**AWS Nitro**
first full-system virtualization	*binary translation on x86*	*paravirtualization*	*hardware-assisted (root/non-root)*	*virtualization as kernel featu*	*hardware offload, custom silicon*

The pattern: *each milestone addresses a limitation of the previous one. The two most recent (VT-x and Nitro) are hardware responses to problems identified by the Popek–Goldberg framework.*

Where we are going

The logical flow of Part I in one picture

Build the machinery
§§1–4

- Motivation
- HW prerequisites
- VM / VMM abstraction
- Direct execution
- Trap-and-emulate

Diagnose the failure
§§5–7

- Privileged vs sensitive
- The HLT case (good)
- The popf case (bad)
- Popek–Goldberg theorem
- x86 fails the test

Resolve the tension
§8

- Binary translation
- Paravirtualization
- Hardware-assisted
- Unified framing
- Trade-offs

The entire lecture is a single arc: we build the tools, use them to spot a problem on x86, and show how the industry solved it in three different ways.

2 Simplified hardware model

A minimum viable computer.

A simplified hardware model

Strip the machine down to what's essential

Real CPUs are complicated. Real ISAs have hundreds of instructions and many privilege levels. To state a devinition clearly, we adopt a minimal abstract model of a computer.

This simplified model has exactly three features:

1

Kernel / user mode bit

A single bit in the CPU that says who is running.

2

Virtual memory

An indirection between virtual addresses (seen by code) and physical memory (in silicon). We use segmentation (reloc + bound) for simplicity.

3

Trap architecture

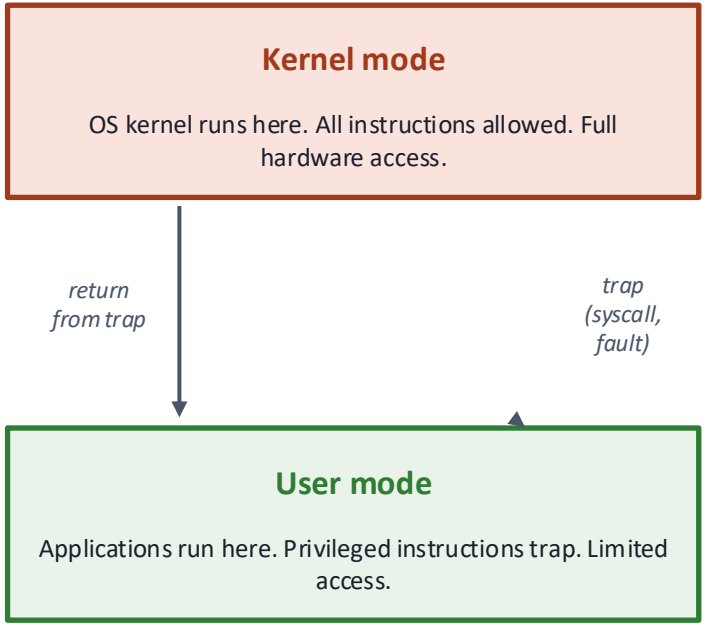
A mechanism to transition user→kernel (syscalls, exceptions) and kernel→user (return from trap).

The privilege bit

One bit: the foundation of OS protection

What it does

- A single hardware bit inside the CPU.
- Two values: kernel mode (0) / user mode (1).
- Determines which instructions are allowed.
- Cannot be flipped by arbitrary code, only by a trap or by explicit return from a trap.
- **Without this bit, the OS cannot protect itself from the applications it runs.**



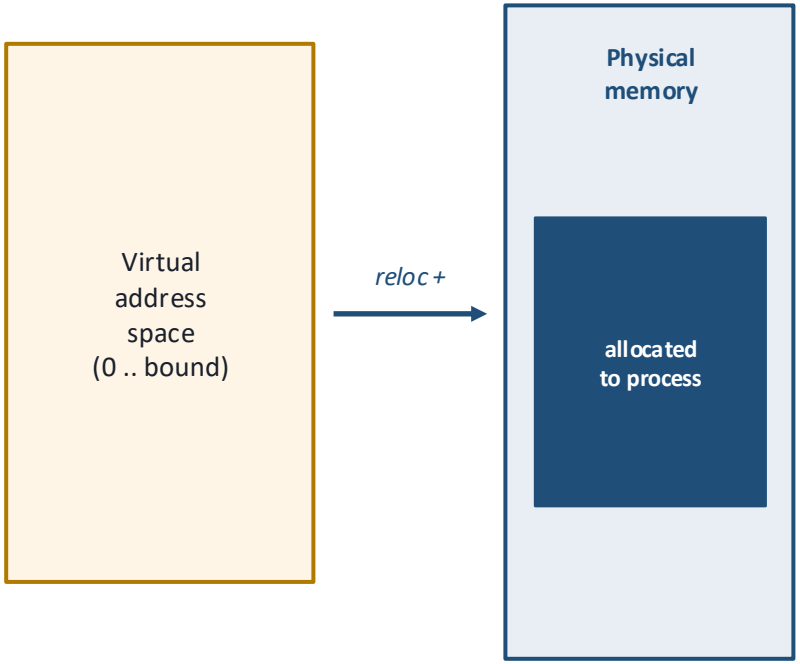
Virtual memory (simplified: segmentation)

Indirection between what the program sees and what is in RAM

- A level of indirection: virtual address → physical address.
- Real hardware uses page tables (4KB pages + MMU). For the lecture, we adopt a simpler model:
- Segmentation, just two privileged registers:

  → *relocation register*: base address in physical memory where virtual 0 is mapped.

  → *bound register*: size of the accessible region.
- virtual address va accesses physical memory at reloc + va, if va < bound.
- **Both registers must be privileged: if user code could change them, it could break out of its sandbox.**



The trap architecture

How the CPU transitions between user mode and kernel mode

Three kinds of trap on any realistic CPU:

System call

Application voluntarily asks the kernel for service (e.g. read, write, mmap).

`int 0x80 / syscall`

Exception

CPU detects an error and stops execution (divide-by-zero, page fault, invalid opcode).

`#GP, #PF, #DE`

Interrupt

External hardware event (timer, disk, network) preempts current execution.

`IRQ 0 (timer)`

Every trap does the same three things:

1. Switches privilege to kernel mode (flips the bit).
2. Transfers control to a pre-registered kernel handler.
3. Saves enough CPU state to resume the interrupted code later.

Direct execution

Applications get the best performance…

The OS does NOT interpret Applications instructions one-by-one.

Instead, Apps instructions execute directly on the physical CPU.

- An app runs directly on the CPU in user mode.
- The OS only intervenes on trap (system call, fault, interrupt).

Why this matters: *direct execution solves the performance requirement: as long as we can also guarantee equivalence and safety.*

Minimal hardware model: summary

With these three features, a protected OS can exist

PROTECTED OS = privilege bit + virtual memory + trap architecture

Each feature is necessary. Remove anyone:

- No privilege bit → applications can overwrite the OS.
- No virtual memory → applications can read/write each other's memory; no isolation.
- No trap architecture → the OS cannot regain control once an application is running.

Hold this model in your head. We will now build a VMM on top of it.

3

Virtual machines and the three requirements

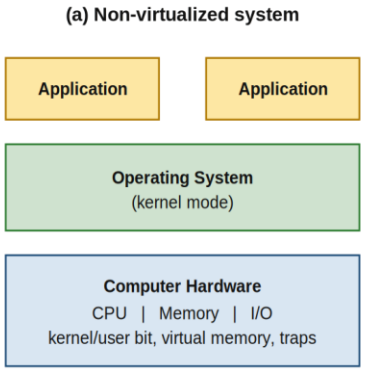
What must a VMM guarantee?

Virtual machine: the abstraction

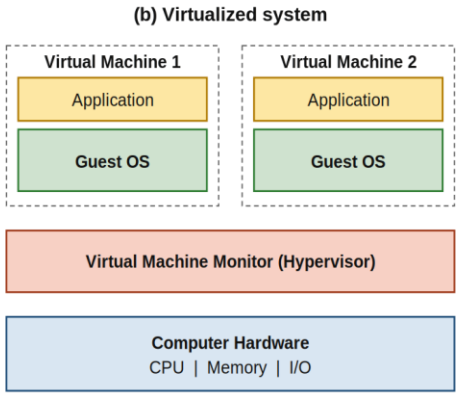
A black box that looks like a computer

A virtual machine (VM) is an abstraction.

- Its interface is sufficiently equivalent to the real hardware that:
  - an operating system can run inside it unmodified;
  - applications can run on that OS unmodified.
- Multiple VMs can coexist on one physical machine.
- The software entity that materializes the VM abstraction is called:
  - Virtual Machine Monitor (VMM) is the classical term;
  - hypervisor (the modern, equivalent term).



OS directly controls HW; applications run in user mode.



The VMM materializes the VM abstraction and mediates access to HW. Each VM runs a guest OS with its own applications, isolated from the others.

Three requirements of a correct VMM

Popek & Goldberg (1974): equivalence, safety, performance

1

Equivalence

Software inside the VM must behave as on the bare hardware. Observationally, the guest cannot tell it is virtualized.

2

Safety

VMs are isolated from each other and from the VMM itself. No VM can observe, interfere with, or compromise another.

3

Performance

Overhead must be small enough that the VM is usable in practice. No interpretation of every instruction.

Each captures a distinct property. The three are not independent: a naive approach can trivially satisfy any two but not all three.

Why this is hard

Each pair of requirements is easy but all three simultaneously is not

Equivalence + Safety ✓	**violates Performance**
Interpret every instruction in software	*10–100× slower than native. Unusable in practice.*
Equivalence + Performance ✓	**violates Safety**
Run the guest OS directly in kernel mode	*One misbehaving guest crashes the host and the other VMs.*
Safety + Performance ✓	**violates Equivalence**
Expose a simplified, non-equivalent ABI (e.g. Java VM)	*Cannot run unmodified OS or arbitrary guest software.*

The central question: can all three be satisfied simultaneously? And if so, how?

4

Direct execution and trap-and-emulate

The canonical VMM construction.

Direct execution

The idea that breaks the performance/equivalence trade-off

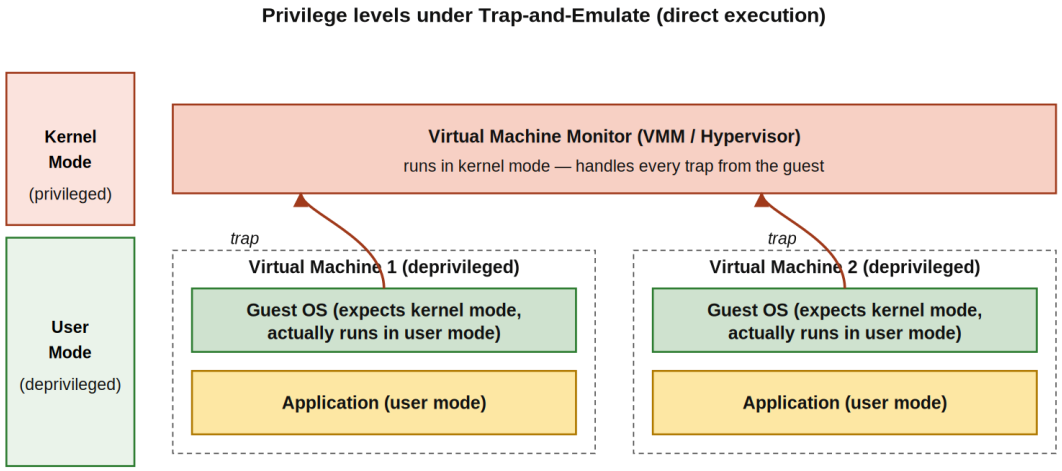
- **The VMM does NOT interpret guest instructions one-by-one.**
- **Instead, guest instructions execute directly on the physical CPU.**

- Analogy: this is exactly how a normal OS runs applications.
  - An app runs directly on the CPU in user mode.
  - The OS only intervenes on trap (system call, fault, interrupt).
- **Virtualization generalizes this: the VMM is to the guest OS what the OS is to its applications.**

Why this matters: *direct execution solves the performance requirement, as long as we can also guarantee equivalence and safety.*

Deprivileging the guest

The VMM rearranges privilege levels



The arrangement:

VMM runs in kernel mode.

Guest OS is forced to run in user mode though it thinks it is still in kernel mode: ***deprivileged***

Guest applications run in user mode (as normal).

Consequence:

When the guest OS tries to execute a privileged instruction (like it normally would in kernel mode), it generates a trap because it is actually in user mode. The VMM (running in kernel mode) catches the trap and emulates the instruction.

Trap-and-emulate

The canonical VMM construction

Loop:

1. **Guest runs directly on CPU**
   in user mode, at native speed. Regular instructions execute normally.
2. **Guest attempts a privileged instruction**
   e.g. disable interrupts, load a segment register, write control register.
3. **Hardware traps (#GP fault)**
   because the guest is in user mode. Trap is routed to the VMM.
4. **VMM emulates the instruction**
   updates its internal model of the guest's virtual state. No real privileged op executes.
5. **VMM returns control to guest**
   guest continues from next instruction, unaware that emulation took place.

Only privileged instructions trap. Regular instructions are free, this is what makes the technique fast.

So this works…

…as long as the hardware cooperates.

Does it always?

5

Does trap-and-emulate always work?

A precise question needs a precise classification.

Does trap-and-emulate always work?

The answer depends on a precise property of the hardware

Trap-and-emulate relies on one crucial assumption:

Every instruction that matters for virtualization must trap when attempted in user mode.

To reason about this precisely we classify instructions on two orthogonal axes:

Axis 1 - HARDWARE

How does the CPU react to the instruction in user mode?

- Privileged: traps in user mode, executes in kernel mode.
- Non-privileged: executes in either mode without trapping.

Axis 2 - SEMANTICS

Does the instruction interact with the machine's control state?

- Sensitive: reads or writes (or depends on) control state.
- Regular: neither touches nor depends on control state.

Privileged instructions

A hardware property, not a semantic one

Definition

An instruction is privileged if it traps when attempted in user mode and executes normally in kernel mode.

This is a HARDWARE fact: it is incised in silicon by the ISA designer.

It has nothing to do with what the instruction does semantically.

On x86:

- HLT is privileged (halts the CPU so hardware decided to trap it).
- LIDT, LGDT are privileged (load descriptor tables).
- MOV to control register is privileged.

An instruction is privileged because the architect wanted it to trap - that's all.

§5.2 · Sensitive instructions

A semantic property - does the instruction touch the control state?

Definition

An instruction is sensitive if its behaviour depends on, or modifies, the control state of the machine (mode bit, memory-virtualization registers, interrupt enable, etc.).

Control-sensitive

Reads or writes the control state directly.

Example:

Reads the mode bit. Writes the relocation register.

Behavior-sensitive

Its semantics depend on the control state, without reading it.

Example:

Returns the physical address for a given virtual address: the result depends on reloc, even if reloc is never read.

Sensitivity is a SEMANTIC fact about what the instruction does - completely independent of whether it is privileged.

The two classifications, side by side

Instructions classified on two orthogonal axes

Privileged vs non-privileged: a hardware fact. Sensitive vs regular: a semantic fact.

HARDWARE: how does the CPU react in user mode?

SEMANTICS: does the instruction touch the control state?

	Privileged *(traps on user-mode execution)*	**Non-privileged** *(executes in user mode, no trap)*
Sensitive *(control- or behavior-sensitive)*	**GOOD CELL** Guest attempts it → hardware traps → VMM emulates. Equivalence preserved. `HLT` — halt the CPU *Privileged: #GP in user mode.* *VMM emulates: park vCPU until virtual IRQ.* `LIDT, LGDT` — load descriptor tables *Privileged: trap and emulate correctly.*	**BAD CELL — violates Popek–Goldberg** Guest modifies / observes control state WITHOUT the VMM ever being invoked. `popf` — pop flags (incl. IF, a control bit) *Not privileged: executes silently in user mode, ignores writes to IF. VMM is never told.* **Silent failure. See popf trace figure.** `sgdt, sidt, smsw, str` *Leak control state in user mode without trapping.*
Regular *(neither control- nor behavior-sensitive)*	**Rare case** Privileged but semantically regular. Traps unnecessarily — pure overhead. `INVLPG, WBINVD` — cache/TLB mgmt *Privileged: always trap in user mode.* *Semantically regular: don't leak state.* **Cost: unnecessary VMEXITs** *(but correctness is preserved).*	**Normal case** The vast majority of instructions. Run at native speed in both modes. `ADD, MOV, JMP, CMP, ...` *Arithmetic, data movement, control flow.* *Neither touch control state nor require trapping.* **No VMM involvement needed.** *The reason direct execution is fast.*

Popek–Goldberg condition: for a VMM to exist, the "BAD CELL" must be empty.

On x86 pre-2006, MIPS, ARM pre-VE, the bad cell is non-empty → trap-and-emulate alone cannot virtualize them.

6

HLT and popf: two concrete examples

The good cell and the bad cell, up close.

Example: HLT the well-behaved case

Sensitive AND privileged → trap-and-emulate works perfectly

What HLT does

- Halts the CPU until the next interrupt.
- Used by idle OS loops to save power.
- Changes control state (CPU activity).
- Sensitive? **Yes.**
- Privileged? **Yes (raises #GP in user mode).**

Under trap-and-emulate

- Guest OS executes HLT → #GP trap → VMM.
- VMM recognises the instruction.
- VMM emulates → parks the virtual CPU until a virtual interrupt arrives.
- **Guest resumes; from its perspective, HLT worked normally.**

Verdict: equivalence preserved.

The guest cannot distinguish virtualized execution from bare-metal execution. HLT sits in the GOOD CELL of the matrix: sensitive AND privileged. Trap-and-emulate handles it perfectly.

Example: popf what it does

A small x86 instruction with a toxic design choice

popf - pop flags

- Pops a word from the top of the stack into the EFLAGS register.
- EFLAGS mixes two very different kinds of flags:

  → arithmetic flags (ZF, CF, SF, OF): harmless, set by nearly every instruction;

  → **control flags in particular IF, the interrupt enable bit: privileged.**
- Used everywhere in code: the standard pattern is pushf / … / popf to save and restore the arithmetic flags around a modifying operation.

The x86 design dilemma (1980s):

- If popf traps in user mode every time → kills performance (popf is used ubiquitously by applications).
- If popf lets user code change IF → security breach (user can disable interrupts).
- **x86's compromise: in user mode, popf executes silently and IGNORES writes to IF.**

popf under trap-and-emulate: the silent failure

The silent failure of popf under trap-and-emulate

Kernel critical-section pattern: pushf / cli / … / popf (guest OS deprivileged in user mode)

	Instruction executed by guest	What the guest OS believes	What the VMM tracks
1	**pushf** (save flags on stack)	Not privileged on x86 → executes in user mode. Reads real EFLAGS, pushes onto stack. `virtual-IF = 1 (interrupts enabled)`	Not involved — no trap. VMM model unchanged. `VMM.virtual-IF[guest] = 1`
2	**cli** (disable interrupts)	Privileged instr. in user mode → #GP. Guest does not see the trap directly; it will resume believing cli succeeded.	VMM intercepts trap, emulates cli: **`VMM.virtual-IF[guest] := 0`** (stop delivering virtual IRQs to guest)
3	**`... critical section ...`** (shared-data update)	Regular user-mode instructions: arithmetic, memory access, no traps. **`virtual-IF = 0 (guest in critical section)`**	Consistent with guest: **`VMM.virtual-IF[guest] = 0`** No IRQs delivered — correct.
4	**popf** (restore flags from stack, intending to re-enable IRQs) **NO TRAP**	popf is NOT privileged on x86. Executes silently in user mode. Restores arithmetic flags (ZF, CF, SF)… **…but silently IGNORES writes to IF.** **`guest BELIEVES: virtual-IF = 1 [WRONG]`**	**VMM NEVER INVOKED.** No trap fires; VMM has no way to know popf attempted to change IF. **`VMM STILL TRACKS:`** **`VMM.virtual-IF[guest] = 0 [WRONG]`**
5	**(subsequent execution)** Guest expects interrupts, waits on I/O, timer, etc.	**`Guest thinks IF=1, IRQs enabled.`** Timer handler should run every 10ms. Scheduler should preempt long tasks. Disk I/O completion should wake tasks.	**VMM still thinks IF=0.** **Suppresses every virtual IRQ.** **No timer. No preemption.** **Guest hangs or deadlocks silently.**

From step 4 onward, guest and VMM disagree about interrupt state — silently.

No crash. No error. No exception. Popek–Goldberg equivalence is violated in the worst possible way: invisibly.

Why popf is the pedagogical example

It concentrates every element of a Popek–Goldberg violation

1. **Sensitive**

   It writes IF, a bit of the architectural control state. Equivalent implementations must observe or reproduce it.

2. **Non-privileged**

   In user mode it executes silently instead of trapping: the worst possible choice for a VMM.

3. **In the BAD CELL of the matrix**

   Exactly the cell that Popek–Goldberg forbids. Sits where no trap-and-emulate VMM can handle it.

4. **Causes silent divergence**

   No crash, no exception, no warning. Guest and VMM disagree; the guest locks up or misbehaves invisibly.

5. **Representative, not contrived**

   pushf / cli / critical section / popf is the canonical kernel pattern. Every real OS uses it.

Next: what does this imply mathematically? The theorem.

7

The Popek–Goldberg theorem

A diagnostic, not a prescription.

§7 · The Popek–Goldberg theorem

Communications of the ACM, Volume 17 / Number 7, July 1974

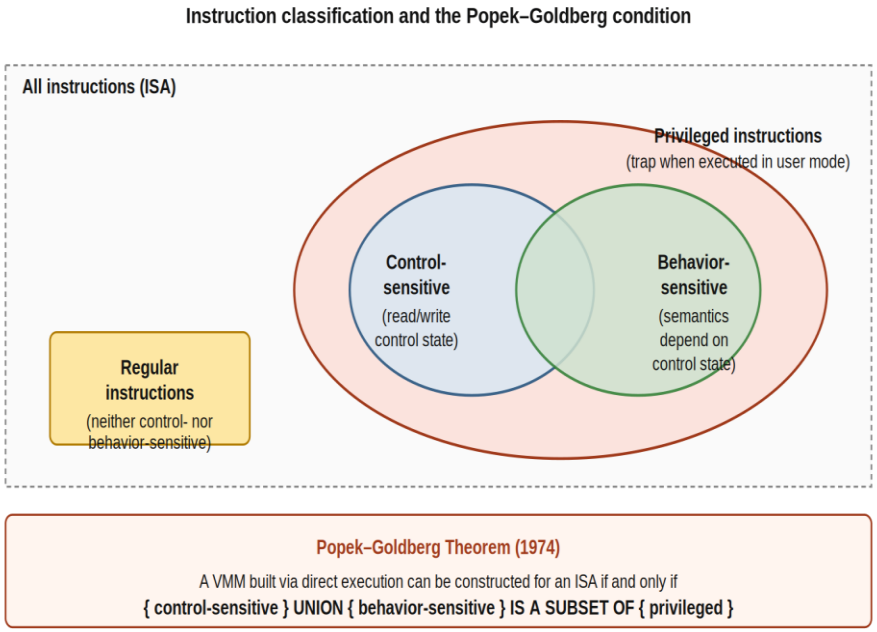
THEOREM

A trap-and-emulate VMM can be built for a given ISA

if and only if

every sensitive instruction is privileged.

(i.e. control-sensitive ∪ behavior-sensitive ⊆ privileged)



Informally: if something "sensitive" can escape user mode without trapping, the VMM will never know and equivalence fails.

Why the theorem is true

Intuitions for both directions

Necessity (⇐)

"if the condition fails, no VMM exists"

- Counterexample: popf.
- Sensitive (writes IF) but not privileged.
- Executes silently in user mode.
- VMM is never invoked.
- Guest and VMM diverge.
- **Equivalence violated.**

Sufficiency (⇒)

"if the condition holds, a VMM can be constructed"

- **Proof by construction:**
  - VMM runs in kernel mode.
  - Guest runs deprivileged in user mode.
  - Every sensitive instruction → privileged → traps → VMM emulates.
  - Memory virtualized by a second level of indirection.
- **The resulting VMM satisfies equivalence, safety, performance.**

The diagnostic applied to real ISAs

Who passes Popek–Goldberg and who doesn't?

ISA	PG condition	Problematic instructions	Consequence
x86 (pre-2006)	**FAIL**	*popf, sgdt, sidt, smsw, … sensitive but not privileged*	Cannot use pure trap-and-emulate.
MIPS (classic)	**FAIL**	*several sensitive instructions non-trapping in user mode*	Cannot use pure trap-and-emulate.
ARM (pre-VE)	**FAIL**	*several sensitive instructions non-trapping in user mode*	Cannot use pure trap-and-emulate.
IBM z/Architecture	**PASS**	*designed with virtualization in mind since the 1960s*	Trap-and-emulate works natively.
PowerPC	**PASS**	*sensitive set ⊆ privileged set*	Trap-and-emulate works natively.

The most popular ISAs of the last 40 years fail Popek–Goldberg. So how did VMware, Xen, KVM actually virtualize them? → §8.

8

Three answers to Popek–Goldberg

Binary translation, paravirt, hardware-assisted.

Three answers to the same question

What do we do with sensitive-but-not-privileged instructions?

Three answers to Popek–Goldberg: when is the fix applied?

Each technique places the "fix" at a different stage of the stack's lifetime

time of intervention

- **CPU design-time** *(silicon)*
- **Guest OS compile-time** *(kernel source)*
- **VMM runtime — pre-exec** *(code translation)*
- **CPU runtime — exec** *(per instruction)*

Hardware-assisted

Intel VT-x, AMD-V — 2006

The CPU is extended with a new orthogonal mode (root / non-root).

Sensitive instructions cause a VM-exit into the VMM.

`popf → VM-exit, VMM emulates.`

Paravirtualization

Xen — 2003

The guest OS is modified at source level. Sensitive code is rewritten as explicit hypercalls to the VMM. Requires source access.

`popf → replaced by HYPERCALL_set_flags.`

Binary translation

VMware — 1998

Guest runs unmodified, but the VMM scans & rewrites code dynamically before execution.

Problematic instructions are replaced with safe sequences.

`popf → rewritten as call to VMM emulator.`

Pure trap-and-emulate

the Popek–Goldberg baseline

No pre-processing. The guest runs directly; sensitive instructions must trap into the VMM by hardware design. **Only works if the ISA is Popek–Goldberg compliant.**

`popf on x86 → fails.`

Trade-offs along the axis:

requires new silicon	requires guest source	requires complex VMM	requires compliant ISA
guest unchanged; near-native performance; dominant since 2006	*no new hardware; near-native performance; lives on in virtio I/O*	*guest unmodified (Windows!); 5–20% overhead; largely obsolete post-2006*	*guest unmodified; elegant but insufficient for x86, MIPS, ARM pre-VE*

Binary translation: VMware, 1998

Fix the guest's code at runtime, before it executes

The idea

- Guest runs unmodified (can be Windows, proprietary OS, any binary).
- VMM scans guest code in basic-block chunks before executing.
- Problematic instructions (e.g. popf) are replaced with safe sequences that call into the VMM.
- Translated blocks are cached and chained for speed.
- Regular user-mode applications code is typically NOT translated (runs directly).

popf under binary translation

Before (in guest binary):

```
popf
```

After translation (runs instead):

```
call vmm_emulate_popf
  ; VMM receives it,
  ; updates virtual-IF,
  ; returns to translated
  ; code stream.
```

Binary translation trade-offs

Powerful but complex

Pros	Cons
• Guest is completely unmodified - supports Windows, closed OSes. • No hardware requirements beyond Popek–Goldberg-non-compliant ISA. • Works on x86 of 1998 no waiting for new silicon. • Founded VMware as a company.	• Enormous VMM engineering complexity (translator, cache, chaining). • 5–20% overhead on compute; higher on system-intensive workloads. • Translation cache pollution on context switches. • Largely obsolete after VT-x (2006) for CPU virtualization.

Historical significance: proved that x86 virtualization is possible, paved the way for cloud computing.

Paravirtualization: Xen, 2003

Modify the guest so it cooperates with the VMM

The idea

- Guest OS is modified at source level to be VM-aware.
- Sensitive operations are NOT issued as native instructions anymore.
- Instead, the guest makes explicit hypercalls into the VMM, like system calls, but to the hypervisor.
- VMM provides a well-defined API for the guest to request privileged services.
- **Guest and VMM collaborate: no guesswork, no translation, no silent failures.**

popf under paravirtualization

In the guest source code (before):

```
push flags
popf
```

After modification (guest calls the hypervisor):

```
HYPERCALL_set_flags(flags)
  /* VMM is told directly
     and can update its
     model consistently. */
```

Paravirtualization trade-offs

Elegant but needs the guest's source code

Pros	Cons
• Near-native performance (no translation, no silent failures). • VMM is dramatically simpler than a BT engine. • No hardware requirements. • Still alive today in I/O: virtio is paravirt applied to devices.	• Guest OS must be modified: works for Linux, hard for Windows (historically impossible). • Guest binary is no longer "standard": cannot run unmodified ISOs. • Tied to a specific hypervisor API (Xen, KVM): portability concern. • Superseded for CPU virtualization by VT-x; persists for I/O where it still wins.

Historical significance: showed cloud-friendly paravirt patterns; virtio (for I/O) remains the industry standard today.

Hardware-assisted: Intel VT-x, 2006

Change the silicon: make Popek–Goldberg true by construction

The idea

- Intel (and AMD with AMD-V) introduced a new, orthogonal CPU mode: root vs non-root.
- Guest OS runs in non-root ring 0: it BELIEVES it is in kernel mode, because for most instructions it is.
- But sensitive instructions (even privileged ones) trigger a VM-exit from non-root to root (the VMM).
- **No need to deprivilege the guest: a whole new protection axis is introduced.**

popf under VT-x

- Guest executes popf in non-root ring 0.
- The CPU is programmed to trigger a VM-exit on popf from non-root mode.
- Control transfers to the VMM in root mode.
- VMM emulates popf semantics consistently.
- VMM issues VM-entry to resume the guest.
- **Popek–Goldberg compliance is obtained in silicon.**

VT-x: more than just "making PG true"

A structurally new protection axis

The new mode (root / non-root) is ORTHOGONAL to the old one (kernel / user).

	Root mode (VMM)	Non-root mode (guest)
Ring 0 (kernel)	**VMM proper** The hypervisor runs here.	**Guest kernel** Guest OS runs here and thinks it's in kernel mode.
Ring 3 (user)	**(rarely used)** VMM user-mode helpers.	**Guest apps** Guest applications in normal user mode.

Since 2006: binary translation dies out for CPU virtualization. HW-assisted is the default on every modern hypervisor.

The three techniques side by side

Comparison along the dimensions that matter

	Binary translation	Paravirtualization	HW-assisted (VT-x)
Year introduced	1998	2003	2006
Who applies the fix	VMM at runtime (pre-exec)	Guest OS developer (source-level)	CPU designer (silicon)
Guest modified?	**No**	**Yes**	**No**
New hardware?	**No**	**No**	**Yes**
Performance	5–20% overhead	Near-native	Near-native
VMM complexity	Very high	Moderate	Low
Status today	*Historical / fallback*	**Alive in virtio I/O**	**Industry standard**

Some Definitions

To know for the final exam ☺

Virtualization

*It is the ability to **simulate a hardware platform**, such as a server, storage device or network resource, in software. All of the functionality is separated (abstracted) from the hardware and simulated as a "**virtual instance**" with the ability to operate just like the hardware solution. A single hardware platform can be used to support multiple virtual devices or machines, which are easy to spin up or down as needed.*

Virtual Machine

It is the software simulation of a computer. It is able to run an Operating Systems and applications interacting with the virtualized abstracted resources, not with the physical resources, of the actual host computer.

Hypervisor (or Virtual Machine Monitor)

It is a software tool installed on the physical host system to provide the thin software layer of abstraction that decouples the OS from the physical bare-metal. It allows to split a computer in different separate environment, the Virtual Machines, distributing them the computer resources

Virtualization Features

Five layers, each building on the previous

Consolidation

Run different OSes on the same physical host Partition physical resources between VMs

Isolation

Fault and security isolation at the hardware level Preserve performance with advanced resource control

Encapsulation

Save the VM state to files VMs can be moved and copied moving and copying files (migration)

Hardware Independence

VMs can be copied, moved or migrated to different physical servers

Other

Debugging, profiling, reproducibility, software preservation

Recap of Part I

The key takeaways to carry into Part II

1. **VM = abstraction, VMM = implementation.**
   Three requirements: equivalence, safety, performance. Each pair is achievable; all three together is the hard problem.
2. **Trap-and-emulate is the canonical VMM construction.**
   Guest is deprivileged; the VMM intercepts and emulates sensitive operations: provided the hardware cooperates.
3. **Popek–Goldberg tells us when "the hardware cooperates."**
   Condition: every sensitive instruction must be privileged. Diagnostic tool, not prescription.
4. **x86 pre-2006 (and MIPS, ARM pre-VE) fail the condition.**
   popf is the canonical counterexample (silent failure, not crash, is the reason the condition is so brittle).
5. **Three industry responses: BT, paravirt, HW-assisted.**
   Each applies the fix at a different point in the stack's lifecycle (design-time, compile-time, or runtime).
6. **In Part II: the same three ideas reappear for I/O.**
   Emulation ↔ BT. virtio ↔ paravirt. Passthrough / SR-IOV ↔ HW-assisted. Same framework, different axis.

End of Part I

Questions?

→ **Next: Part II Resource virtualization in practice**

I/O virtualization, storage, network, HPC performance considerations